

# Validation Report

## Budget Greenfield Architecture — North Star (Ledger Rebuild)

*Independent architecture & best-practice validation*

Prepared 31 May 2026 · Scope: ledger correctness, concurrency, GoBD/audit, migration safety

---

|                                                  |   |
|--------------------------------------------------|---|
| 1. Executive summary .....                       | 2 |
| 2. Best-practice alignment scorecard .....       | 2 |
| 3. What the plan gets right (keep these) .....   | 3 |
| 4. Findings by area .....                        | 3 |
| 4.1 Ledger correctness .....                     | 3 |
| 4.2 Concurrency .....                            | 4 |
| 4.3 GoBD / audit .....                           | 5 |
| 4.4 Migration safety .....                       | 5 |
| 5. Hardening recommendations .....               | 6 |
| 6. Additions to the invariant & test spec .....  | 8 |
| 7. Suggested adjustments to the phase plan ..... | 8 |
| 8. Bottom line .....                             | 9 |
| Sources .....                                    | 9 |

## 1. Executive summary

The north-star document is, on the whole, a strong and well-reasoned design. Its central thesis — collapse four conflicting “used” definitions into one read formula, materialize credits as real rows, and remove derivation from read paths — is squarely in line with how mature financial-ledger systems are built. The proposed hold / consumed / released / reversed lifecycle is a textbook two-phase reservation model and maps almost one-to-one onto the pending / posted / voided semantics used by purpose-built ledger engines. If executed as written, it would resolve the class of bugs you have been fighting (anchor-flooring, double-count guards, drift between surfaces).

That said, the plan is **not yet bulletproof**. It has four gaps that, left unaddressed, will reproduce the ledger instability under a different name: (a) it is a *balance-equation* model, not a true double-entry model, so nothing structurally forces the books to sum to zero; (b) the hold→consumed reconciliation can silently change *which pot* is charged, not just the amount; (c) overdraft protection at *plan time* is asserted but not architecturally enforced against concurrency; and (d) holds and tax-relevant bookings are conflated in one GoBD-governed table. None are fatal — each has a concrete fix below.

**Verdict:** Approve the direction; do not start Phase 4 (hard-holds) until the concurrency-enforcement and hold/consume reconciliation items (R3, R4, R5) are designed in. The 17 findings and 12 recommendations below, plus 6 added invariants, are what turn this from a good plan into a defensible one.

## 2. Best-practice alignment scorecard

Each design pillar is rated against current (2025–26) ledger-engineering guidance. “Aligned” means the plan already matches best practice; “Gap” means a change is recommended.

| Design pillar                                            | Status  | Notes                                                                                                                        |
|----------------------------------------------------------|---------|------------------------------------------------------------------------------------------------------------------------------|
| Materialize credits as rows; derive balances by SUM      | Aligned | Matches the consensus that balances must be derived from immutable entries, never stored as a mutable column that can drift. |
| One read formula across all surfaces (SSoT)              | Aligned | Single-reader / CQRS read-model is the standard cure for “N definitions of used.”                                            |
| Hold → consumed → released/reversed lifecycle            | Aligned | One-to-one with two-phase (pending/posted/voided) transfers in dedicated ledger engines.                                     |
| planCascade as one pure function for holds + consumption | Aligned | Determinism + purity is exactly what makes split logic property-testable.                                                    |
| Expand → backfill → contract migration with equality net | Aligned | Parallel-change pattern; equality/golden tests before deleting legacy is the right safety oracle.                            |
| True double-entry (sum-to-zero) enforcement              | Gap     | Plan is a balance equation, not balanced entries. Add a conservation/reconciliation invariant (R1).                          |
| Overdraft prevention under concurrency                   | Gap     | Check-then-write at plan time is a TOCTOU race unless serialized or DB-enforced (R3).                                        |

| Design pillar                              | Status | Notes                                                                              |
|--------------------------------------------|--------|------------------------------------------------------------------------------------|
| Hold/consume reconciliation semantics      | Gap    | Re-running cascade on actuals can change the pot, not just the amount (R4).        |
| Holds vs. tax-relevant bookings separation | Gap    | Holds are not GoBD bookings; one shared immutable table over-constrains them (R8). |
| Reservation timeout / orphan sweep         | Gap    | No expiry on holds; stuck appointments reserve budget forever (R6).                |
| Idempotency keys on ledger writes          | Gap    | Not specified; required to make retries/double-clicks safe (R5).                   |
| Production shadow-read soak before cutover | Gap    | Equality tests cover a corpus; add live dual-run drift monitoring (R10).           |

### 3. What the plan gets right (keep these)

These decisions are correct and should be protected through implementation reviews so they are not eroded under deadline pressure:

- **“Every euro is a row, nothing derived in reads.”** This is the right north star. Critically, the plan derives *Available* via a SUM aggregate over rows rather than storing a balance column — which is exactly the best practice (a cached balance is the classic drift source). Guard against anyone later “optimizing” by adding a materialized balance column.
- **The two-phase reservation lifecycle.** hold/consumed/released/reversed is the same shape as pending/posted/voided in dedicated ledger engines, where a reservation provably cannot push an account past its balance invariant and can be posted-or-voided exactly once. Adopt those two guarantees explicitly (see R3, R5).
- **One pure planCascade for both holds and consumption.** Sharing the split function is what kills preview/booking drift. Keep it pure and side-effect-free so property tests stay meaningful.
- **Private/Selbstzahler as an “infinite pot.”** Expressing policy as pot composition instead of branches is elegant and removes a whole class of special-case bugs. One caveat in R7 about how “infinity” is represented.
- **Equality net before deleting legacy.** Capturing legacy outputs across a customer matrix and requiring byte-identical reproduction is the correct migration safety oracle.

### 4. Findings by area

Findings are grouped by the four areas you prioritized and tagged by severity. Recommendation IDs (R1...R12) are defined in section 5.

#### 4.1 Ledger correctness

| Severity | Finding                                                                                                                                                                                                                                                                                                 | Fix |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| HIGH     | F1 — Not true double-entry. The model is a one-sided ledger plus a balance equation ( $\text{Available} = \text{Allocated} - \text{Holds} - \text{Consumed}$ ). Nothing structurally forces total debits to equal total credits, so a bug in one write path can leave the system internally consistent- | R1  |

| Severity        | Finding                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Fix    |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|                 | looking but wrong. Mature ledgers enforce that every transaction sums to zero across accounts.                                                                                                                                                                                                                                                                                                                                                             |        |
| <b>CRITICAL</b> | F2 — Hold/consume can change the pot, not just the amount. Completion re-runs planCascade on documented actuals. If actuals exceed the planned estimate, the consumed split may spill into a different (or the private) pot than the hold reserved — or require a pot that didn't exist at plan time. The plan flags "delta reconciliation" but does not define semantics for a pot-changing delta or an over-budget completion of already-performed work. | R4     |
| <b>HIGH</b>     | F3 — State machine omits the correction path. I3 allows only consumed→reversed. Editing a completed appointment to a new value needs consumed→reversed and then a fresh $\emptyset$ →consumed (or $\emptyset$ →hold→consumed). The allowed-transition test will reject the legitimate correction unless this compound path is specified.                                                                                                                   | R2     |
| <b>MEDIUM</b>   | F4 — Lazy \$45b row creation "on read" writes inside the read path. This contradicts "nothing derived/written in reads," and two concurrent reads can race to insert the same monthly row. Relying only on a uniqueness constraint turns the race into user-visible errors.                                                                                                                                                                                | R3, R9 |
| <b>MEDIUM</b>   | F5 — "Infinite pot" arithmetic. Available(private) = $\infty$ must never leak into sums, reports, or cents math (overflow, polluted totals). Storing a literal huge amountCents is fragile.                                                                                                                                                                                                                                                                | R7     |

## 4.2 Concurrency

| Severity        | Finding                                                                                                                                                                                                                                                                                                                                                                                               | Fix |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>CRITICAL</b> | F6 — Overdraft race at plan time. Planning is read-Available-then-write-hold: a classic time-of-check/time-of-use race. Two concurrent planning operations on the same (pot, period) can both observe budget and both write holds, overcommitting the pot. The plan keeps "advisory-lock serialization of consumption," but holds are a NEW write path at plan time and are not covered by that lock. | R3  |
| <b>HIGH</b>     | F7 — Double-transition / double-complete. Completing the same appointment twice (retry, double-click, duplicate job) can write two consumed rows. A reservation must be post-or-void exactly once.                                                                                                                                                                                                    | R5  |

| Severity | Finding                                                                                                                                                                                                                                                                          | Fix     |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| HIGH     | F8 — No idempotency keys. No deduplication key is specified for plan/complete/cancel. Retries are unsafe by default in any at-least-once delivery path (queues, webhooks, client retries).                                                                                       | R5      |
| MEDIUM   | F9 — Series materialization write-amplification. Writing up to 365 hold rows per series at plan time, plus lazily creating future-period allocation rows, concentrates lock contention and can serialize behind the same (customer, pot, period) lock used for overdraft safety. | R6, R12 |

### 4.3 GoBD / audit

| Severity | Finding                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Fix    |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| HIGH     | F10 — Holds and tax-relevant bookings are conflated. A hold reserves budget but moves no money; it is an operational reservation, not a Buchung. Putting holds, releases, consumes and reversals in one GoBD-immutable table makes routine plan/cancel churn look like booking/storno activity, creating audit noise and over-constraining an operational concern. GoBD immutability and storno rules should bind the financial layer (consumed/reversed), not reservations. | R8     |
| HIGH     | F11 — “State-close” is an UPDATE. Closing a hold by flipping a state column is a mutation. GoBD requires that changes are never silent (nicht unbemerkt) and only via documented storno/adjustment. A flag flip must be append-only or trigger-logged, or it is a compliance gap.                                                                                                                                                                                            | R8, R9 |
| MEDIUM   | F12 — No Verfahrensdokumentation deliverable. GoBD (updated by BMF letters of 11 Mar 2024 and 14 Jul 2025) requires process documentation a knowledgeable third party can follow. This north-star is a good seed but is not yet that document.                                                                                                                                                                                                                               | R11    |
| MEDIUM   | F13 — Temporal correctness of the audit trail. Holds/bookings with a past transactionDate must bind to the settings and allocations valid then, including which immutability rules applied at that time. Good that I10 exists; extend it to the reversal/correction path.                                                                                                                                                                                                    | R2, R8 |

### 4.4 Migration safety

| Severity | Finding                                                                                                                                                     | Fix |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| HIGH     | F14 — Phase 4 before Phase 5 creates a split-brain window. Hard-holds go live in Phase 4 but the unified reader lands in Phase 5. In between, some surfaces | R10 |

| Severity | Finding                                                                                                                                                                                                                                      | Fix      |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| HIGH     | subtract holds and others don't — reproducing exactly the “different surfaces disagree” bug class you are trying to kill. Reads should be unified before, or in the same release as, holds becoming visible.                                 |          |
|          | F15 — Equality net is corpus-only. A fixed corpus can miss live-data shapes. Add production shadow-reads (compute legacy and new for every real read, log diffs) and require a zero-drift soak window before any legacy deletion in Phase 6. | R10      |
| MEDIUM   | F16 — Backfill mechanics unspecified. “Forward-only” is decided, but batching, checkpointing, pause controls, and a post-backfill reconciliation query are not. Unbatched backfills lock tables and spike IO on live data.                   | R12      |
| MEDIUM   | F17 — No per-phase rollback statement. Each phase “ships,” but there is no explicit “legacy path remains callable and is the source of truth until soak passes” guarantee, nor a documented revert for each phase.                           | R10, R12 |

## 5. Hardening recommendations

Twelve concrete changes, ordered roughly by leverage. Treat R3, R4, and R5 as gating for Phase 4.

### R1 — Add a conservation/reconciliation invariant (move toward double-entry)

You do not have to refactor into full debit/credit accounts overnight, but you should add the guarantee double-entry gives you for free: a periodic (and test-time) reconciliation that, for every (pot, period), the sum of allocation rows minus the sum of active holds minus net consumed equals the independently computed Available, and that every consumed amount is fully accounted for by allocation draw-downs. Run it as a read-only verifier in production and as a property test in CI. Longer term, consider modeling the private/insurer side as real counter-accounts so transactions are required to sum to zero at write time.

### R2 — Specify the full state machine, including corrections

Document every legal transition, not just the happy path:  $\emptyset \rightarrow \text{hold}$ ,  $\text{hold} \rightarrow \text{consumed}$ ,  $\text{hold} \rightarrow \text{released}$ ,  $\text{consumed} \rightarrow \text{reversed}$ , and the compound correction  $\text{reversed} \rightarrow (\text{new}) \emptyset \rightarrow \text{hold}/\text{consumed}$ . Make the architecture test assert this exact set and reject everything else. Define what happens to holds when an appointment is rescheduled across a period boundary (release in old period, hold in new).

### R3 — Enforce non-negativity at the database, not in application checks

Replace check-then-write with a guarantee the database cannot violate. Three viable options, in order of robustness: (1) a per-(customer, pot, period) advisory lock taken for the whole plan-and-write transaction so holds are serialized the same way consumption already is; (2) *SERIALIZABLE* isolation with retry-on-serialization-failure on the planning transaction; or (3) a balance-invariant enforced in the database itself (constraint/trigger) that refuses any hold or consume that would drive a capped pot negative — the same “debits must not exceed credits” guarantee dedicated ledger engines apply at reservation time, so the second phase can never break the invariant either. Prefer (1)+(3): serialize the hot path AND make overdraft structurally impossible. Note the \$45b lazy-create must take the same lock or use an idempotent upsert (ON CONFLICT DO NOTHING).

#### **R4 — Define hold/consume reconciliation precisely (the pot-change problem)**

Decide and document the rule for when documented actuals differ from the plan: (a) if actuals  $\leq$  hold, post the actual and release the remainder of the hold (partial capture); (b) if actuals  $>$  hold and the additional amount still fits the same pots under the cascade, extend within the same pots; (c) if the overflow would require a different or the private pot, this is a policy event — specify whether it auto-spills to private (for private-capable customers) or raises a typed “over-budget completion” that an operator resolves. Do not silently re-pot already-performed work. Capture this as golden tests (hold split vs. consumed split) for each case.

#### **R5 — Idempotency keys + post-or-void-once enforcement**

Attach an idempotency key to every ledger-mutating operation (plan, complete, cancel), unique per (appointmentId, occurrence, intendedTransition). A unique constraint makes a duplicate complete a no-op rather than a second consumed row. Enforce that a given hold can be consumed or released exactly once (a closed hold rejects further transitions), mirroring the post-or-void-once rule of two-phase ledger transfers.

#### **R6 — Give holds a lifetime / orphan sweep**

A reservation with no expiry can sit in limbo forever if an appointment is never completed or cancelled. Tie hold lifetime to the appointment lifecycle and add a read-only reconciliation sweep that flags holds whose appointment is in a terminal-but-uncleared state, so reserved budget is never lost. (Dedicated ledger engines attach an explicit timeout to pending transfers precisely for this reason.)

#### **R7 — Model the private pot as “uncapped,” not a literal infinity**

Represent the private pot with an explicit uncapped flag rather than a giant amountCents. The cascade treats an uncapped pot as “absorbs all remainder”; reporting and totals must special-case or exclude it so  $\infty$  never enters sums, averages, or cents arithmetic. Add an invariant: no report or aggregate ever reads a numeric capacity from an uncapped pot.

#### **R8 — Separate the operational reservation layer from the GoBD financial ledger**

Keep two conceptual layers even if they share a table: holds/released are operational reservations (not Buchungen, not tax-relevant); consumed/reversed are the financial, GoBD-immutable bookings. Apply the immutability triggers, storno semantics, and retention rules to the financial layer only. This removes audit noise from routine planning churn, avoids treating a released reservation as a booking storno, and keeps the Finanzamt-facing trail clean. If kept in one table, mark which states are GoBD-relevant and scope triggers accordingly.

#### **R9 — Make state transitions append-only (or trigger-logged)**

Prefer recording each transition as a new append-only row referencing its predecessor over mutating a state column in place. This is the purest GoBD posture (changes are never silent) and it also sidesteps the lazy-create and double-transition races. If you must mutate a state column, every change must fire an immutability/audit trigger that records who/when/from→to.

#### **R10 — Production shadow-read soak + reorder reads before holds**

Two changes to the rollout: (1) Unify the read SSoT before or in the same release as hard-holds becoming visible, so no window exists where surfaces disagree about available-after-planned. (2) Before deleting any legacy path in Phase 6, run new and legacy in parallel on live reads (shadow mode), log every per-customer diff, alert on non-zero drift, and require a clean soak window — not just a green corpus — as the deletion gate. Keep the legacy path the source of truth until that gate passes; that is your rollback.

#### **R11 — Produce a Verfahrensdokumentation for the budget subsystem**

Elevate this north-star into a formal process documentation that a knowledgeable third party (auditor) can follow: data flow, how reservations differ from bookings, how corrections/storno work, retention, and the immutability controls. Reference the current GoBD basis (BMF letters of 11 Mar 2024 and 14 Jul 2025). This is both a compliance requirement and excellent onboarding material.

## R12 — Specify backfill and series materialization mechanics

For the §45b backfill: batch (e.g., low-thousands of rows), checkpoint, expose pause/resume, and run a reconciliation query comparing materialized SUMs to legacy at multiple as-of dates before switching reads. For series: bound or asynchronously materialize the 365-occurrence holds so a single series creation does not hold the hot lock for an extended write, and make future-period allocation creation idempotent.

## 6. Additions to the invariant & test spec

The plan's I1–I12 are a strong oracle set. Add these six to close the gaps above.

| ID  | Invariant                                                                                                                                                                                                             | How to assert                                                                   |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| I13 | Conservation / reconciliation: for every (pot, period), independently summed Allocated – HoldsActive – ConsumedNet equals the reader's Available, and every consumed cent traces to allocation draw-down.             | Read-only prod verifier + CI property test (R1).                                |
| I14 | Overdraft impossibility: no interleaving of concurrent plan/complete operations can drive a capped pot below zero.                                                                                                    | Concurrency/stress test with parallel writers; DB-constraint test (R3).         |
| I15 | Post-or-void-once: a hold can transition to consumed or released at most once; duplicate operations are no-ops.                                                                                                       | Idempotency test: replay the same operation N times, assert single effect (R5). |
| I16 | Reconciliation semantics: for actuals $\leq$ hold, $>$ hold same-pots, and $>$ hold cross-pot, the consumed split matches the documented rule; cross-pot over-budget raises the typed event, never silent re-potting. | Golden tests per case (R4).                                                     |
| I17 | Reservation/financial separation: released holds never appear in GoBD/financial exports; only consumed/reversed do.                                                                                                   | Export/audit snapshot test (R8).                                                |
| I18 | No-drift soak: live shadow-read drift is zero across the soak window before any legacy deletion.                                                                                                                      | Production shadow-read dashboard + deletion gate (R10).                         |

## 7. Suggested adjustments to the phase plan

1. **Pull read unification earlier.** Move the single-reader work (current Phase 5) to land before or with hard-holds (current Phase 4), eliminating the split-brain window (F14, R10).
2. **Gate Phase 4 on R3/R4/R5.** Do not ship hard-holds until overdraft enforcement, reconciliation semantics, and idempotency are designed and tested.
3. **Gate Phase 6 on a shadow-read soak, not just tests.** Legacy stays the source of truth until live drift is zero for the soak window (F15, R10).



4. **Add a Phase 0.** Decide the reservation-vs-financial-ledger split (R8) and the state-transition storage model (append-only vs. trigger-logged, R9) before building tables — these are hard to change later.

## 8. Bottom line

The architecture is heading in the right direction and the test-spec discipline is better than most production ledgers ever get. The remaining risk is concentrated in four places: enforce non-negativity structurally (not by application check), define what happens when documented reality diverges from the reservation, separate operational reservations from tax-relevant bookings, and prove equivalence on live data before deleting anything. Address R3, R4, R5, R8, and R10 and this stops being a plan that merely re-locates your ledger problems and becomes one that ends them.

## Sources

- [Double-Entry Ledgers: The Missing Primitive in Modern Software — Paul Gross](#)
- [Enforcing Immutability in your Double-Entry Ledger — Modern Treasury](#)
- [How to Scale a Ledger, Part V: Immutability and Double-Entry — Modern Treasury](#)
- [Books, an immutable double-entry accounting database service — Square](#)
- [Two-Phase Transfers — TigerBeetle Docs](#)
- [Transfer reference \(pending/posted/voided, timeouts\) — TigerBeetle Docs](#)
- [Authorization Holds: A Guide for Businesses — Stripe](#)
- [Place a hold on a payment method \(capture before, expiry\) — Stripe Docs](#)
- [Idempotency in Distributed Transaction Systems — ByteDoodle](#)
- [Fixing Race Conditions in PostgreSQL for Financial Systems — Daniel Dallas Okoye](#)
- [PostgreSQL: Transaction Isolation \(Serializable\) — PostgreSQL Docs](#)
- [The race condition a stress test found in my double-entry ledger — DEV](#)
- [Event Sourcing Pattern — Microsoft Azure Architecture Center](#)
- [Database Migrations: The Expand-Contract Pattern — Enol Casielles](#)
- [Zero-Downtime Database Migrations for TypeScript SaaS — AverageDevs](#)
- [GoBD: Grundsätze ordnungsmäßiger Buchführung — Lexware](#)
- [Neuerungen der GoBD-Richtlinien 2025 \(BMF 11.03.2024 / 14.07.2025\) — Datenschutzberater.NRW](#)
- [Ordnungsgemäße Buchführung: GoBD — IHK München](#)